01_데모_시연_대본

데모 시연 대본 — CompaniAI v1.1

> 청중: 민덕기 교수님 + NLP/에이전트 랩실 팀원 데모 시간 목표: 5–7분 (시연 본 트랙 mock 38–45초 + 설명 시간) 보조 자료: PPT 슬라이드(03), 연구계획서(02)

---

0. 시연 전 사전 준비 (발표 시작 30분 전)

환경 점검

```
# 좀비 프로세스 청소
pkill -f "main.py --dashboard" ; pkill -f vite

# 포트 비어있는지 확인
lsof -nP -iTCP:8000 -sTCP:LISTEN
lsof -nP -iTCP:5173 -sTCP:LISTEN

# RAM 압박 줄이기 (큰 앱 종료) — 진짜 모드 보여줄 가능성 있다면 권장
osascript -e 'quit app "Microsoft Teams"'
osascript -e 'quit app "Notion"'
osascript -e 'quit app "Discord"'

# 데모 시작
cd "/Volumes/ 개발(휴대)/프로젝트/compani_ai_dev3"
./demo
```

브라우저 사전 점검

- 데모 자동 오픈된 페이지 새로고침 → 헤더 우상단 토글이 **MOCK · 시연 모드 (38초)** 인지 확인
- “Dashboard online” 배지 초록색
- Stage Card 5개 모두 ○ pending 상태
- 다운로드 폴더 비우기 (자동 다운로드 시 깔끔하게 보이게)

절대 클릭 금지 (발표 중)

- 토글 (실제 LLM 모드로 가지 마세요. 16GB Mac 환경에서 hang 위험)
- 만약 잘못 클릭했다면 → confirm 모달의 “취소 (Mock 모드 유지)” 누르기

---

1. 도입 — 30초

발표 멘트 (예시)

> "오늘 시연할 시스템은 **CompaniAI** 라는 멀티에이전트 기반 소프트웨어 자동 생성 시스템입니다.

> 핵심 아이디어는, **사용자가 자연어로 아이디어 한 줄을 던지면 CTO 에이전트가 전략을 수립하고, 작업을 분해해서, 역할별 SLM 에이전트들이 협업하며 실제 동작하는 코드를 만들어내는 구조** 입니다. 모든 모델이 로컬 Ollama 로 동작해 비용 0, 16GB Mac 환경에서 재현 가능합니다. 지금 바로 보여드리겠습니다."

가리키는 곳

- 화면: 데모 페이지 헤더 (“CompaniAI Live Demo”)
- “MOCK 시연 모드 (38초)” 토글 손가락으로 가리킴 → 청중에게 “이 시연은 사전 정의된 시나리오로 안전하게 보여드리고, 실제 동작은 5–10분 걸리지만 같은 흐름입니다” 라고 명시

---

2. 입력 — 20초

진행

1. 추천 프리셋 첫 번째 **“간단한 Todo REST API 만들어줘”** 클릭
2. textarea 에 자동 입력된 것 보여줌
3. (잠깐 멈춤) “보시다시피 자연어 한 문장입니다. 어떤 기술 스택을 쓸지, 파일을 몇 개 만들지, 누가 무엇을 할지 — 사용자는 아무것도 지정하지 않습니다.”
4. **제출하기** 클릭

발표 멘트

> “보시다시피 사용자는 ‘Todo API 만들어줘’ 한 문장만 입력했습니다. 이제부터 CTO 에이전트가 이걸 어떻게 분해하고, 5개의 task를 어떤 에이전트들이 어떤 협업을 하며 처리하는지 실시간으로 보시게 됩니다.”

---

3. 파이프라인 진행 — 25초 (자동 진행)

진행 (입력 후 0–5초)

- “CTO 전략 수립” 카드가 파란 glow 펄스로 활성화
- 카드 안에 라이브 sub-status: “qwen3:8b 추론 호출 (전략 생성)”

발표 멘트 (이 시점)

> “**1단계 — CTO 전략 수립.** 지금 화면에 빛나고 있는 카드를 보시면, qwen3:8b 모델이 사용자 요청을 분석해 ‘FastAPI + React + Docker’ 라는 기술 전략을 수립하고 있습니다. 실제 환경에서는 이 단계만 30초에서 1분 정도 걸리지만, 시연에서는 압축해서 보여드립니다.”

5–10초

- “CTO 전략 수립” 완료
- “CTO 작업 분해” 카드 활성화 → “5개 task 생성: backend 2 / frontend 1 / mlops 2”

발표 멘트

> "**2단계 — 작업 분해.** CTO가 전략을 5개의 구체적인 task로 분해했습니다. 보시면 backend 2개, frontend 1개, mlops 2개로 역할이 자동 할당됐습니다. **이 분해 결과를 LLM이 직접 만든다는 점이 중요한데**, 이게 멀티에이전트 시스템의 첫 번째 자율성입니다."

10–25초

- "에이전트 병렬 실행" 활성화
- 아래 Task 진행 상세 표가 실시간으로 채워짐
- T-001 backend (FastAPI CRUD), T-002 frontend (React UI), T-003 mlops (Docker) 동시에 running

발표 멘트 (Task 표 가리키며)

> "**3단계 — 에이전트 병렬 실행.** 지금 표를 보시면 세 에이전트가 동시에 일을 하고 있습니다. 화면 왼쪽에는 어떤 task가 어떤 모델로 돌고 있는지가 보이고, 오른쪽에는 경과 시간이 실시간으로 갱신됩니다. **이 병렬성이 중요한데** — 각 에이전트가 LLM 응답을 기다리는 동안 다른 에이전트가 일할 수 있어서 전체 처리 시간이 단축됩니다. Python asyncio 로 구현했습니다."

---

4. 협업 시각화 — 핵심 차별점 (90초)

이 부분이 발표의 클라이맥스입니다. **천천히** 설명하세요.

진행 (메시지 로그에 항목들이 등장하기 시작)

- 첫 QA 메시지: `backend → frontend: PATCH 형식?`

발표 멘트

> "여기서 중요한 부분이 시작됩니다. 실제 개발팀에서도 그렇듯이, 에이전트끼리도 **서로 질문**을 합니다."

행동

1. 첫 번째 QA 메시지 카드 **클릭** → 모달 등장
2. 모달이 뜨면 **잠깐 멈춤** (2-3초, 청중이 읽을 시간)

발표 멘트 (모달 가리키며)

> "Backend 에이전트가 frontend 에이전트에게 질문했습니다.
>
> **왜 질문했나** — frontend 가 소비할 PATCH 응답 형식을 사전에 합의하기 위해.
>
> **질문 내용** — JSON Patch (RFC 6902, op 기반) 와 JSON Merge Patch (RFC 7396, 단순 객체) 중 무엇이 더 좋은지.
>
> **답변** — frontend 에이전트가 'JSON Merge Patch 권장' 이라고 답하면서 그 **근거** 까지 줍니다. 'React setState 결과를 그대로 직렬화 가능하기 때문' 이라고요.
>
> 이게 바로 단순한 메시지 패싱이 아니라, **컨텍스트와 근거를 동반한 협업**입니다. 그리고 이 대화 내용은 backend 에이전트의 다음 코드 생성 프롬프트에 자동 주입돼서, 실제 결과물이 합의된 형식으로 나옵니다."

모달 닫고 다음 — 회의 메시지 등장

> “다음을 보시면, **회의 열림** 이라는 메시지가 등장했습니다.”

행동

1. 회의 메시지 클릭 → Emergency Meeting 모달

발표 멘트 (모달 천천히)

> "**왜 열렸나** — Backend 산출물에 redis 의존성이 추가됐는데 mlops 의 docker-compose 에 redis 서비스가 빠져있어서, 이대로 두면 컨테이너 빌드 실패가 예상되는 상황이었습니다. 시스템이 자동으로 충돌을 감지하고 회의를 소집했습니다.
>
> **참석** — CTO, Backend, MLOps 세 에이전트입니다.
>
> **대화 흐름** — mlops 가 먼저 문제를 지적하고, backend 가 대안을 제시하고, **CTO가 최종 결정**을 내립니다. ‘둘 다 가능하지만 캐시는 메트릭 가시화에도 도움되니 redis service 추가하는 방향으로’ 라고요. 발언자별로 색깔이 다른 좌측 보더로 누가 무엇을 말했는지 한눈에 보입니다.
>
> 이 회의의 결과로 mlops 에이전트의 task 가 자동 업데이트됩니다."

모달 닫고 — 피어 리뷰 메시지

> “마지막으로, 피어 리뷰 메시지가 등장했습니다.”

행동

1. 피어 리뷰 메시지 클릭 → 모달

발표 멘트

> "T-001 의 backend 산출물을 frontend 가 리뷰합니다. **왜 frontend 가 리뷰하나** — frontend 가 그 API를 소비하는 입장이니까요.
>
> 코멘트 두 개가 있습니다. - 하나는 **MINOR** 등급으로, ‘POST 응답이 200 OK 인데 REST 컨벤션상 201 Created 가 적절하다’ 는 권고입니다. - 다른 하나는 **INFO** 등급으로, Pydantic schema 분리가 깔끔하다는 칭찬과 함께 frontend types 으로 그대로 옮길 수 있겠다는 코멘트입니다.
>
> Verdict 는 **PASSED** 이지만 권고 사항 한 건은 후속 patch 로 처리. 이게 사람이 GitHub 에서 PR 리뷰하는 흐름과 거의 동일합니다."

---

5. 산출물 확인 — 60초

진행 (38–45초 시점)

- 5단계 모두 완료
- 결과 카드 등장: exit_code = 0
- 자동 다운로드 완료 — `todo-rest-api-mock.zip` 다운로드 폴더에

발표 멘트

> "이제 파이프라인이 끝났습니다. **15개 파일이 생성됐고, 자동으로 zip 다운로드까지 됐습니다.** 결과 카드를 보시면 어떤 task가 몇 초 걸렸는지, 평균 처리 시간이 얼마인지 모두 표시됩니다."

행동

1. **산출물 미리보기** 버튼 클릭 → 트리 등장
2. 트리에서 `backend/main.py` 클릭 → 우측에 코드 미리보기

발표 멘트 (코드 가리키며)

> "FastAPI 엔트리 포인트입니다. 보시면 lifespan 관리, CORS 설정, router 분리, healthz 엔드포인트까지 — **단순한 hello world 가 아니라 실제 production 수준의 backbone** 이 들어가 있습니다."

행동

1. `backend/routers/todos.py` 클릭

발표 멘트

> "라우터 코드를 보시면 권고 사항이 반영되어 **POST 가 status_code=201_CREATED** 로 돼있고 (피어 리뷰 결과), **PATCH 가 JSON Merge Patch** 형식으로 구현되어 있습니다 (QA 결과). 즉 에이전트들이 주고받은 대화가 실제 코드 결정에 영향을 미친 흔적이 명확하게 남아있습니다."

행동

1. `frontend/src/api.ts` 클릭

발표 멘트

> "그리고 frontend api 클라이언트입니다. PATCH 함수가 backend 가 합의한 형식으로 보낼 수 있게 `TodoPatch` interface 를 정의하고 있습니다."

---

6. 마무리 — 30초

발표 멘트

> "정리하면, 이 시스템은 다음 세 가지를 보여줍니다:
>
> 1. **자율성** — 사용자가 한 문장 던지면 CTO 가 전략, 분해, 할당까지 자율 처리
> 2. **협업의 깊이** — 단순한 메시지 패싱이 아니라 **이유 있는 질문, 합의 회의, 코드 리뷰** 가 자동화
> 3. **추적 가능성** — 모든 결정의 근거가 산출물 코드까지 이어지는 trace 형태
>
> 그리고 이 시스템을 운영하면서 식별된 두 가지 한계 — **에이전트 간 통신 프로토콜이 최적인지** 와 **반복되는 실수를 어떻게 기억할지** — 가 본 연구의 RQ 출발점입니다. 이어서 연구 질문으로 넘어가겠습니다."

예상 질문 + 답변 가이드

A. 시스템 아키텍처 질문

Q1. ChatDev 과 무엇이 다른가요?

답변: > "ChatDev 는 같은 LLM(GPT-4)을 역할만 바꿔서 반복 호출하는 구조입니다. 즉 Backend 에이전트도 Frontend 에이전트도 같은 GPT-4 인스턴스에 다른 system prompt 를 줘서 시뮬레이션합니다. > > 제 시스템은 **모델 자체를 다르게** 씁니다. CTO 는 qwen3:8b 같은 reasoning 모델, Backend/Frontend 는 gemma4:e4b 같은 코드 생성 특화 SLM, MLOps 는 llama3.2:3b 같은 가벼운 instruction-following 모델 — 역할에 맞는 모델을 매칭합니다. 모든 모델이 로컬 Ollama 에서 돌아서 비용이 0이고 16GB Mac 에서 재현 가능합니다. > > 그리고 ChatDev 는 통신을 자연어 토큰 그대로 주고받는데, 저는 SharedWorkspace + MessageQueue 의 구조화된 통신을 사용합니다. 그래서 통신 토큰이 ChatDev 보다 적게 들고, 무엇보다 — 그게 RQ2 의 출발점인데 — 어떤 통신 프로토콜이 정확성과 속도를 동시에 만족하는지 비교 실험할 수 있습니다."

Q2. MetaGPT 와는?

답변: > "MetaGPT 는 SOP(Standard Operating Procedure) 기반으로 워크플로우가 사전 정의되어 있습니다. PRD → 시스템 디자인 → API 디자인 → 코드 순으로 고정된 단계를 따라갑니다. > > 제 시스템은 CTO 가 task decomposition 을 LLM 추론으로 동적으로 결정합니다. 즉 'Todo' 같이 단순한 요청은 5개 task, '블로그 플랫폼' 같이 복잡한 요청은 12개 task로 자동 늘어납니다. 그리고 의존성도 동적이라 dep_completed 시점에 다음 task 가 시작됩니다."

Q3. 왜 SLM 인가요? GPT-4 같은 LLM 한 개가 더 효율적이지 않나요?

답변: > "이게 RQ1 의 핵심 질문이고, 실은 검증이 필요합니다. 제 가설은 **task 가 잘게 나뉘고 역할 전문성이 중요한 경우** SLM 팀이 같은 FLOPs 에서 LLM 단일보다 우위가 있다는 것입니다. > > 직관은 이렇습니다. GPT-4 한 번 호출은 비용이 크지만 매번 모든 컨텍스트를 처리합니다. 반면 SLM 5개를 병렬로 부르면 각자는 자기 task 컨텍스트만 처리하니 빠르고 가볍습니다. 단, 통신 오버헤드가 추가됩니다. > > 실험으로 확인하고 싶은 것은 어느 task 유형에서 break-even point 가 있는가입니다. ProtocolBench 2025 처럼 동일 FLOPs 조건으로 비교 실험을 설계하려고 합니다. 솔직히 결과는 모릅니다 — 가설 검증이 연구의 목적입니다."

B. 통신 프로토콜 (RQ2) 질문

Q4. Protocol D 의 importance_score 가중치(0.4×관련성 + 0.3×최신성 + 0.3×심각도)는 어떻게 정했나요?

답변: > "현재는 **임의의 초기값**입니다. 직관적으로 '관련성이 가장 중요하지만, 최근 정보와 심각한 정보도 무시할 수 없다' 라는 정도로만 잡은 hand-tuned 시작점입니다. > > 연구 과정에서 두 가지를 할 계획입니다: > 1. **그리드 서치** — 가중치 조합을 0.1 단위로 변화시키며 task completion rate 와 통신 토큰 수의 trade-off 측정 > 2. **학습 기반** — 과거 task 결과 데이터셋이 쌓이면 가중치 자체를 메타러닝으로 최적화 > > 다만 RQ2 의 본질은 가중치 자체보다 **'중요도 기반 선택적 통신' 이라는 패러다임이 자연어 전체 전송보다 효율적인가** 입니다. 가중치는 그 안의 implementation detail 입니다."

Q5. ProtocolBench 와 차별화 포인트는?

답변: > "ProtocolBench 는 기존 프로토콜들을 비교하는 벤치마크입니다. 제가 거기 더하고 싶은 건 **(a) Protocol D 의 새로운 제안** 과 **(b) 집단 메모리와의 연계** 입니다. > > 특히 (b) 가 중요한데, 통신 프로토콜과 메모리는 사실 분리된 문제가 아니라고 봅니다. 어떤 정보를 어떻게 주고받느냐에 따라 메모리에 쌓이는 데이터의 품질도 달라집니다. 두 axis 를 같이 보는 연구는 아직 많지 않은 것 같습니다."

C. Collective Error Memory (RQ3) 질문

Q6. ExpeL 과 무엇이 다른가요?

답변: > "ExpeL 은 단일 에이전트의 경험 누적입니다. 한 에이전트가 task 들을 풀어가면서 자기 경험을 메모리에 쌓고 다음 task 에 활용합니다. > > 제가 하고 싶은 건 **팀 전체가 공유하는** 집단 메모리입니다. 예를 들어 backend 에이전트가 'Pydantic v2 에서 `from_orm` 대신 `model_validate` 써야 한다' 는 실수를 한 번 했다면, 다음에 frontend 에이전트가

type 정의 작업을 할 때도 그 정보를 사용 가능해야 합니다. > > 그리고 한 단계 더 나아가서, 같은 실수가 N번 이상 반복되면 **자동으로 방지 규칙으로 승격**해서 다음 에이전트의 system prompt 에 고정 삽입하는 방식을 실험하고 싶습니다. ExpeL 의 insight extraction 과 비슷하지만 팀 단위 + 자동 규칙화입니다."

Q7. 어떤 데이터를 메모리에 저장하나요? Hallucination 같은 것은 어떻게 식별?

답변: > "현재 설계는 **(a) 명시적 실패** 와 **(b) 암묵적 실패** 두 가지를 구분합니다. > > (a) 명시적 실패는 에이전트가 던진 ErrorCode (`E-LLM-TIMEOUT` , `E-PARSE-JSON` , `E-DEPS-BLOCKED` 등) — 이건 자동 캡처됩니다. 코드/severity/context 함께 Qdrant 에 임베딩. > > (b) 암묵적 실패는 어렵습니다. peer review 에서 reviewer 가 지적한 코멘트를 활용할 계획입니다. severity MAJOR 이상 코멘트는 'author 가 이 부분에서 실수했다' 의 신호로 간주. > > 진짜 hallucination — 즉 reviewer 도 못 잡은 잘못된 정보 — 는 어렵습니다. 이건 사용자 피드백이나 외부 검증 (예: 생성된 코드의 실제 테스트 통과 여부) 으로 보완해야 합니다."

Q8. 메모리가 noisy 하게 쌓여서 오히려 false positive 를 일으키지는 않을까요?

답변: > "정말 중요한 질문입니다. 현재는 두 가지로 방어를 생각합니다. > > 1. **임계값** — 최소 N번 (예: 3번) 이상 같은 패턴이 반복되어야 '방지 규칙'으로 승격. 한두 번 실수는 noise 로 간주. > 2. **Confidence score** — 각 메모리에 confidence 를 매겨서, 검색 시 임계값 이상만 프롬프트에 주입. > > 다만 솔직히 말씀드리면 이 부분은 실제 데이터 없이는 튜닝이 어렵습니다. 실험 단계에서 noise injection 도 의도적으로 해서 robustness 를 확인할 계획입니다."

D. 실험 / 검증 질문

Q9. 어떤 도메인에서 실험하나요? compani_ai 만으로는 generalizability 가 부족하지 않나요?

답변: > "타당한 지적입니다. 두 도메인을 병렬로 진행할 계획입니다. > - **compani_ai** — 코드 생성 (제가 만든 시스템이라 실험 통제 쉬움) > - **OLaLA** — 가짜뉴스 판별 (작년 다른 task 형태로 같은 SLM 협업 검증) > > 다만 두 도메인만으로는 부족합니다. 가능하면 HumanEval 같은 표준 벤치마크나 ProtocolBench 의 task suite 도 추가로 검증하고 싶습니다. 다만 이건 실험 환경 (16GB Mac) 의 한계로 task scope 가 제한될 수 있어, 교수님 지도가 필요한 부분입니다."

Q10. 16GB Mac 으로 어떻게 학술적 의미 있는 실험이 가능한가요? GPU 클러스터 없이?

답변: > "한계 인정합니다. 다만 두 가지 관점입니다. > > 1. **재현 가능성 강조** — 16GB Mac 환경 자체가 contribution 일 수 있습니다. 'High-end GPU 없이도 멀티에이전트 시스템이 실용적인가' 라는 질문은 실무자/오픈소스 커뮤니티에 의미가 있습니다. > > 2. **연구실 자원 활용 가능성** — 만약 연구실에서 더 큰 자원을 사용할 수 있다면 GPT-4 baseline 비교나 더 큰 모델 (Qwen-2.5 70B) 사용도 가능합니다. 16GB Mac 은 prototyping 환경으로 두고, 본 실험은 연구실 자원 활용을 희망합니다."

E. 시스템 / 구현 질문

Q11. SharedWorkspace + MessageQueue 구조의 합리성은? Database 없이 메모리 dict 로도 가능하지 않나요?

답변: > "두 가지 이유로 SQLite + Qdrant 조합을 선택했습니다. > > 1. **Persistence + Replay** — 파이프라인 중간 crash 가 나도 SQLite 에 저장된 WorkItem 상태로 재개 가능. 디버깅과 재현성에 중요합니다. > > 2. **벡터 검색** — Q3 의 집단 메모리에서 유사 task 검색이 필요한데 Qdrant 가 그 역할입니다. 그리고 Qdrant 의 collection 을 SharedWorkspace 의 query 로 노출하면 자연스럽게 통합됩니다. > > 메모리 dict 는 v0 prototype 일 때 사용했는데, async 환경에서 race condition 잡기 어렵고 persistence 가 없어서 디버깅이 매우 힘들었습니다. SQLite 로 옮기면서 EventBus + WAL 모드 조합으로 동시성 문제도 해결됐습니다."

Q12. 통신은 EventBus 인데 Q&A 는 MessageQueue. 두 채널을 분리한 이유는?

답변: > "두 채널의 의미론이 다릅니다. > > - **EventBus (publish/subscribe)** — '상태 변화 알림'. 누가 듣든 상관없는 broadcast. 예: `task.completed` 는 모든 구독자에게 전파. > - **MessageQueue (point-to-point)** — '특정 에이전트에게 응답을 요구하는 메시지'. Q&A 는 응답을 기다려야 하므로 큐의 dequeue 의미론이 맞습니다. > > 분리하지 않고 EventBus 로 통합할 수도 있지만, 그러면 'Q에 답이 왔는지' 추적을 application layer 에서 다 해야 합니다. SQLite-backed MessageQueue 가 그걸 자연스럽게 처리합니다."

Q13. Failure case 는 어떻게 처리하나요? CTO 가 잘못된 task 분해를 하면?

답변: > “Stage Gate Meeting 이라는 메커니즘이 있습니다. 모든 task 가 끝난 후 (또는 task_count 의 절반이 끝난 후) CTO 가 모여서 ‘success rate 이 임계값 이상인가, avg duration 이 한도 이내인가’ 를 평가합니다. 미달이면 abort 또는 rework. > > 다만 솔직히 말씀드리면 — CTO 가 처음부터 task 분해를 잘못한 경우는 아직 견고하지 않습니다. 시니어 QA 리뷰에서 식별된 R-09C (rework 시 dependency 재구성) 가 그 예입니다. 대학원 연구에서는 이 부분을 **runtime adaptation** 으로 강화하고 싶습니다 — task 실패 패턴을 메모리에 쌓아 다음 분해 시 CTO 가 참조.”

F. 발표 진행 관련 안전 답변

시연 중 데모가 멈췄을 때

> “아, 이게 mock 모드로 시연하는데도 가끔 timing 이 어긋날 수 있습니다. 잠시 새로고침 한 번 하겠습니다.” (F5 → 토큰 URL 다시 입력 또는 페이지 reload → mock 다시 제출)

진짜 모드를 보여달라는 요청

> “교수님, 진짜 LLM 호출 모드는 16GB Mac 환경에서 5–10분 걸리고 swap 발생 가능성이 있어서, 발표 중 완주를 보장하기 어렵습니다. 대신 이전에 진짜로 돌렸을 때의 산출물을 보여드릴 수 있고요, 또는 발표 후 별도 시간에 함께 보시는 게 어떨까요?”

답을 모르는 질문

> “솔직히 그 부분은 아직 깊이 파보지 않았습니다. 추정으로는 [핵심 직관 한 줄] 이라고 생각하는데, 정확한 답을 드리려면 [관련 논문/실험] 을 더 봐야 합니다. 대학원에서 같이 풀어보고 싶은 부분이기도 합니다.” (모르는 척하지 말고 정직하게 — 랩실 발표에서는 이게 신뢰도를 높임)

시연 직전 체크리스트

```
[ ] 데모 살아있음 (포트 8000, 5173 listen 확인)
[ ] 헤더 토글 =    MOCK (절대 실제 모드 X)
[ ] Stage 카드 5개 모두 ○ pending (이전 run 잔여 X)
[ ] 메시지 로그 비어있음
[ ] 다운로드 폴더 비움 (Finder → 다운로드)
[ ] Teams / Notion / Discord 등 무거운 앱 종료 (RAM 확보)
[ ] 노트북 화면 sleep 비활성화 (시스템 설정 → 디스플레이 → "절대 안함")
[ ] 화면 배율 적당 (외부 모니터 시 폰트 크기 확인)
[ ] PPT 슬라이드 장표 손에 (다음 단계 안내용)
[ ] 질문 모드: 정직하게, 모르면 모른다
```